
Block 14: A Multi-Agent Recombination Plan, and Why FlashAttention-4 Is Not Our Kernel Lever

Table 5 (hypothesis author), OPHIS¹

Abstract

This is the block hypothesis paper for block 14 of the `walltime_5min_h200` challenge, a precondition for execution per this campaign’s own `EXPERIMENT_WORKFLOW.md` and the operator’s explicit instruction to plan before spending GPU time. Reference baseline 1.0301; current SOTA 0.9506 mean / 0.9483 best. We first correct an operator-supplied prior: FlashAttention-4 (Dao et al., 2026) is co-designed specifically for NVIDIA Blackwell’s tensor-core-to-memory-bandwidth asymmetry, every reported speedup is B200-measured, and our target GPU is H200 (Hopper) already running the Hopper-appropriate FA3 backend – FA4 is not a transferable lever here. A targeted re-fetch of Recursive’s own writeup independently corroborates this: it names no FlashAttention/FA4 lever at all, crediting its $0.9372 \rightarrow 0.9109$ gain to hashed n-gram value-path gating (already our own campaign’s central mechanism), the Muon optimizer, and squared-ReLU activations. We register six new external papers with full claim/evidence chains, most valuably a peer-reviewed TACL 2025 paper showing an n-gram-aware loss regularizer cuts LM memorization by up to 40% – independent, peer-reviewed corroboration of this campaign’s own C1/C2/C3 memorizer-regularization family. We then run a three-proposer, one-adversarial-critic multi-agent round (architecture, systems/kernel, data-efficiency axes) that pools 12 grounded candidates and keeps 7 for a block-14 execution plan, explicitly built around two multi-lever combinations testing opposite

interaction hypotheses – a de-confounded retest of paper 013’s confirmed negative interaction, and a new redundancy test between two mechanistically-related memorizer regularizers – per the operator’s standing directive to prefer recombination over isolated one-off tests. Five candidates are cut with stated reasons, including a explicit refusal to re-litigate the already-rejected C2 composite-hash mechanism family without new information. All 7 kept candidates are registered as mechanisms, hypotheses, interventions, and novelty-checked idea-archive records before this paper is finalized. No GPU time is spent in this paper; it is the plan, not the result.

1. Introduction

Paper 013 closed with five ranked next directions and a methodological lesson: isolated single-lever testing is “a machine for producing increments” that cannot see interactions, and this campaign’s own cycle-5 factorial ($\text{AttnRes} \times \text{n-gram-table weight decay}$) found a real super-additive negative interaction that four prior isolated cycles, run in the same block, entirely missed. Block 14 is the next planning cycle. Two things happened before any new experiment was planned. First, the operator asked us to analyze what Recursive’s automated-research writeup did right, flagging attention-kernel work (specifically FA4) as plausibly important, and separately asked us to read recent DeepSeek and Kimi/Moonshot papers and prefer their ideas. Second, the operator restated, unprompted by a specific result, the standing instruction from paper 013’s own postmortem: prefer coherent multi-lever combinations over testing one change in isolation and discarding it immediately.

This paper does three things in response. Section 2 reports what the FA4 investigation actually found – a correction of the operator’s own prior, not a confirmation, reached by reading the primary source rather than assuming a name-recognition heuristic (“atten-

¹Autoresearch reimplementation campaign. Correspondence to: Fable 5 (OPHIS) <baiyuzhu@mit.edu>.

tion kernel paper” → ”must apply here”). Section 3 reports five other new external sources, most importantly a peer-reviewed corroboration of this campaign’s existing memorizer-regularization track. Section 4 reports the multi-agent propose-and-critique round: three axis-scoped proposers reading the actual code and registries, one adversarial critic that kept 7 of 12 candidates and cut 5 with stated reasons, producing block 14’s execution plan (Table 1).

2. FlashAttention-4 Is Not Our Lever

FlashAttention-4 (Dao, Princeton/Together AI/Meta/NVIDIA/Colfax, released 2026-03-05) is a ground-up attention-kernel redesign reaching up to 1605 TFLOPs/s on B200 (71% utilization), 1.1–1.3× faster than cuDNN 9.13, and up to 3.6× faster than FA2 at 32768-token sequence length. Every one of these numbers is measured on Blackwell. The kernel’s stated design rationale – treating Blackwell’s specific tensor-core-throughput-to-memory-bandwidth ratio as a first-class constraint and co-designing an asymmetric pipeline schedule around it – does not describe Hopper’s different asymmetry profile, and the source reports no H200 measurement at all. We register this as `clm_fa4_blackwell_hardware_coupling` with a literature evidence assessment (`evd_lit_fa4_hopper_scope_mismatch`) whose `trust.scope_match` is explicitly weak: strong, well-documented evidence, applied to the wrong hardware generation for this campaign.

This matters because our existing attention backend is already FA3 – chosen and hardened specifically for Hopper in an earlier block (the campaign’s own docs/Agent_must_read.md records the `segfault-to-custom_op` fix that made FA3 work under `torch.compile fullgraph=True`). There is no version-number argument (“4 > 3”) that survives contact with the actual hardware-coupling in FA4’s design. We independently double-checked the operator’s premise by re-fetching Recursive’s own writeup with a prompt specifically targeting kernel/FA4/throughput content: it names no FlashAttention version, FA4, or dedicated attention-kernel optimization anywhere in the retrievable NanoChat-autoresearch section. The reported gains are architecture-level (hashed bigram/trigram embeddings gated into the attention value path – already this campaign’s own central n-gram-VE mechanism), optimizer-level (Muon), and activation-level (squared-ReLU). We register this as `clm_recursive_no_kernel_lever_reported`. Two independent checks – one hardware-coupling analysis, one targeted re-read of the cited source – both point the

same direction: FA4 is not a transferable lever for this campaign, and the operator-elevated kernel candidate from paper 012 (fp8 attention projections, explicitly a Hopper-native mechanism via `torch._scaled_mm`) remains the correct near-term systems bet, sequenced as block 14’s Section 4 explains.

3. Six New External Sources

Beyond FA4 and the Recursive re-check, we registered four more sources with full `pap_/clm_/evd_lit_chains`. DeepSeek-V4 (arXiv 2606.19348) introduces Manifold-Constrained Hyper-Connections (mHC) and a hybrid CSA/HCA sparse-attention mechanism. We traced mHC to its actual primary source rather than resting on the secondary DeepSeek-V4 summary: Hyper-Connections (Zhu et al., ICLR 2025, arXiv 2409.19606) expands a single residual stream into n learned-mixture streams, but the naive (unconstrained) formulation breaks the residual identity-mapping property and causes training instability at scale; mHC (Xie et al., DeepSeek-AI, arXiv 2512.24880) fixes this via a manifold projection that restores identity mapping. Neither source’s abstract-level fetch surfaced quantitative loss/perplexity deltas, which we record honestly as a limitation rather than inferring a magnitude. CSA/HCA, by contrast, targets million-token-context KV-cache efficiency; by the same profiled-step-time reasoning that led this campaign to reject Kimi Delta Attention in paper 013 (attention is ~5% of step time at our 2048-token, no-cache-reuse regime), we assess CSA/HCA as scope-mismatched and record it as a documented non-candidate rather than silently dropping it.

The most valuable find is Slack & Al Moubayed (TACL 2025, arXiv 2510.11372): fine-tuning Pythia/Llama3/Mistral (1.4B–70B), they show memorization rises sharply in early epochs, often before validation perplexity is optimized, and that an n-gram-aware loss regularizer cuts a measured memorization score by up to 40%. Both findings independently corroborate this campaign’s own from-scratch results at 50M parameters: paper 011’s finding that the n-gram memory is the sole carrier of data-sensitivity, and paper 013’s C3 confirmation (−0.0214 at 5 shards, ~14σ). This is the strongest external, peer-reviewed support yet registered for the C1/C2/C3 track (`evd_lit_ngram_regularizer_corroborates_family`, `assessment.relation: supports`, `strength: strong`).

4. Multi-Agent Propose and Critique

Three proposers, each reading `train.py`, paper 013, the current registries, and this block’s new literature directly (not summarized secondhand), generated 12 candidates across architecture/convergence-bias, systems/kernel/throughput, and data-efficiency/memorizer-regularization. Every candidate was self-scored 1–10 on novelty, validity, impact, reliability, and provenance, with a stated code site and a preregistered falsifier. An adversarial critic then reviewed the pooled slate, revised scores where it disagreed (stating why), and produced a 7-candidate execution order honoring: no two same-RESEARCH DIRECTIONS candidates back-to-back, roughly 60/40 explore/refine, and – per the operator’s explicit directive – a genuine multi-lever combination near the front of the block, not deferred to “if time allows.”

4.1. Kept (execution order 1–7)

(1) Fixed-step isolation of AttnRes (`hyp_block14_attnres_fixedstep_isolation`, refine, architecture). Cycle 5’s AttnRes-alone arm regressed `val_bpb` by +0.0219 but completed only 1438/1956 steps (–26%) – a real throughput confound, not a clean quality verdict. Re-run baseline vs. AttnRes at a matched ~1420-step budget. Zero new code (reuses `int_attnres_full`); scores `nov3/val9/imp8/rel9/prov9` – the highest validity/reliability/provenance candidate in the pool precisely because it introduces no new mechanism, only a controlled re-measurement of an already-registered, currently-held hypothesis.

(2) CUDA-graph capture (`hyp_block14_cudagraph_capture`, refine, systems_kernel). The frame overrides `torch.compile`’s own coded default (max-autotune) down to max-autotune-no-cudagraphs specifically to free ~25GB for larger n-gram tables (`train.py:1796–1806`). Re-enabling capture should collapse the profiled ~194k-tiny-kernel launch-bound sink into single-launch replay – if that sink lives inside the compiled graph. B1 (paper 013) already falsified in-graph fusion as the fix; this tests graph replay instead, a mechanistically distinct claim. The critic downgraded validity from the proposer’s 8 to 7 and added an explicit stop/pivot: `_ngram_sparse_rmsprop_step`’s own docstring (`train.py:1999–2002`) states it runs eager because `torch.unique` has data-dependent shape, incompatible with `fullgraph=True` – structurally excluded from cudagraph capture regardless of outcome here. A null result on this candidate is therefore evidence for candidate 5, not evidence the kernel-sink

question is closed.

(3) Re-run the `AttnRes × C1` factorial at fixed step count with the corrected weak lambda (`hyp_block14_attnres_c1weak_interaction`, explore, architecture, combination). This is the block’s required near-front recombination test. Cycle 5’s factorial found AttnRes (+0.0219) and C1 at $\lambda = 0.2$ (+0.0272) individually harmful, and their combination super-additively harmful (+0.0715 vs. an additive prediction of +0.0491, a +0.0224 excess). But both components were independently fragile: AttnRes lost 26% of its steps to unfused-kernel cost, and this cycle’s own $\lambda = 0.02$ result (`run_c1sweep_wd002`: 0.9619 vs. ref 1.0573 at 5 shards, a huge win) shows $\lambda = 0.2$ was far too strong. We re-run the full 4-arm design at matched fixed step count with $\lambda = 0.02$, reusing candidate (1)’s baseline and AttnRes-alone arms rather than re-running them. The preregistered decision rule: interaction excess within ± 0.003 of zero confirms the original finding was a confound-compounding artifact; excess > 0.010 confirms it persists as a genuine representational conflict. Either answer is a real finding. Scores `nov8/val8/imp8/rel6/prov9`.

(4) C1 weak-lambda bracket at 10 shards (`hyp_block14_c1_10sh_bracket`, refine, memory_retrieval). Paper 013’s ranked direction #2 was only half-executed – $\lambda \in \{0.02, 0.05\}$ at 5 shards (both large wins) but not at the 10-shard operating point, where only the harmful $\lambda = 0.2$ result exists. If harm scales with λ , weak lambda should land in $[-0.005, +0.003]$, possibly promoting C1 to a second flagship data-efficiency lever alongside C3. Zero new code; `nov3/val8/imp6/rel8/prov9`.

(5) Batched cross-table sparse optimizer step (`hyp_block14_sparse_optimizer_batched`, explore, systems_kernel). A genuinely different diagnosis from B1: `_ngram_sparse_rmsprop_step` is called once per table, each paying a full eager 5-op chain (`unique/index_add_/index_select/sqrt/index_add_`), never entering `torch.compile`’s graph. Offset-packing all tables into one combined index space collapses this to one chain total, preserving exact per-row arithmetic by construction. Not yet implemented – registered with `intervention.status=proposed`, gated on a cheap `nsys` pre-check confirming this region is $>10\%$ of the profiled kernel count before implementation effort is spent, per this campaign’s own B2-after-reprofile discipline. Sequenced after (2) so a fresh trace and (2)’s own result jointly inform whether this is the more plausible remaining sink. `Nov7/val8/imp6/rel6/prov8`.

(6) `C1(weak) × C3` redundancy test

(hyp_block14_c1_c3_redundancy, explore, memory_retrieval, combination). The block’s second multi-lever test, deliberately probing the opposite interaction shape from (3). C1 and C3 are two different implementation sites – optimizer-step weight shrinkage vs. forward-activation count gating – of the same diagnosed phenomenon (harm concentrated in low-count n-gram keys). We predict a sub-additive, redundant interaction: the combined 5-shard benefit should beat either lever alone but fall short of their naive additive sum (-0.1168 , from the already-measured -0.0954 and -0.0214). A result worse than either lever alone would be a second, mechanistically distinct negative interaction – evidence the recombination premise does not automatically hold even for related regularizers. Composes two already unit-tested interventions; no new code. Nov7/val7/imp7/rel8/prov9.

(7) Minimal manifold-constrained write-back into x_0 (hyp_block14_mhc_lite_writeback, explore, architecture). The only candidate directly operationalizing this block’s own mHC literature find. x_0 (train.py:1397) is currently assigned once and only ever read via resid_lambdas/ x_0 _lambdas – a one-way tap, not a genuine second residual stream. We give it a learned, softmax-row-normalized (a point on the Birkhoff/doubly-stochastic manifold – literally the “manifold constraint” the literature attributes the HC stability fix to) write-back from each block’s output, zero-init as an exact no-op. Not yet implemented. Scoped deliberately as a minimal pilot (one write-back, a 2×2 matrix) rather than a full multi-stream commitment, per the literature-evidence assessment’s own recommendation, given the residual-stream-addition family’s mixed local track record (AttnRes held, Canon rejected). Nov7/val6/imp6/rel7/prov8.

4.2. Cut, with reasons

Simplex-constrained read-gate (architecture): the proposer’s own estimated effect ($|\Delta| < 0.004$) sits barely above the adoption gate, and it is dominated by candidate (7) on the same literature claim. Standalone fp8-attention pilot (systems_kernel): the proposer’s own text predicts a near-invisible standalone result at the current launch-overhead-dominated operating point (paper 012’s own “masked” framing) – fully subsumed as one arm of the deferred combo below. Cudagraph+fp8 “unmasking” combo (systems_kernel): stacks two first-time-in-repo implementations at once, the lowest reliability self-score (5/10) in the entire 12-candidate pool; deferred, not rejected, pending candidate (2)’s own result. Cross-family C2 composite-hash retry (memory_retrieval): extends

Table 1. Block-14 execution plan. Two combinations (bold) test opposite interaction hypotheses.

#	Candidate	Direction	Ready
1	AttnRes fixed-step	architecture	yes
2	CUDA-graph capture	systems_kernel	yes
3	AttnRes $\times$ C1(0.02)	architecture	yes
4	C1 bracket @10sh	memory_retrieval	yes
5	Batched sparse-opt	systems_kernel	code
6	C1(weak) $\times$ C3	memory_retrieval	yes
7	mHC-lite write-back	architecture	code

meh_ngram_composite_coverage_capacity_tradeoff, a mechanism already registered REJECTED by an explicit two-sided falsifier in paper 013 – the operator’s own screening instruction forecloses spending a gated GPU run on this family without first validating the rescue attempt offline. C3 kappa sensitivity at 10 shards (memory_retrieval): lowest novelty (2/10) and impact (3/10) in the pool; its own governing mechanism predicts the effect it tests for is unlikely to exist at this operating point, and it is dominated by candidates (4) and (6) on the same axis.

5. Discussion

Two process points are worth stating plainly. First, correcting our own operator’s stated prior (FA4) required going to primary/technical sources rather than trusting name recognition – “FlashAttention-4 exists and is newer” is not the same claim as “FlashAttention-4 helps here,” and the two independent checks in Section 2 (hardware-coupling analysis, targeted source re-read) agreed. Second, this block’s plan is built around two combinations testing opposite interaction shapes on purpose: (3) re-examines whether a confirmed destructive interaction was real or confound-driven, while (6) tests whether a constructive-in-kind but potentially redundant pairing shows diminishing returns. Running both in the same block, rather than one now and one someday, is the direct operational answer to the operator’s standing instruction: hold more than one branch alive and test combinations before individual verdicts are treated as final.

6. Conclusion

Block 14 spends zero GPU-seconds and produces: a corrected kernel-priority prior (FA4 does not transfer to Hopper; fp8-attention-projections remains the right near-term bet), six newly registered external sources including a peer-reviewed corroboration of the campaign’s own memorizer-regularization thesis, and a 7-

candidate execution plan built by adversarial multi-agent critique rather than unilateral selection, with two deliberately-opposite-hypothesis multi-lever combinations placed at positions 3 and 6. Round 1 (AttnRes fixed-step isolation) is the immediate next authorization: zero new code, highest-provenance candidate in the pool, and a direct precondition for round 3’s factorial.

References

- Dao, T. et al. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. 2026.
- Recursive. First Steps Toward Automated AI Research. Company blog, 2026.
- Zhu, D. et al. Hyper-Connections. arXiv:2409.19606, ICLR 2025.
- Xie, Z. et al. mHC: Manifold-Constrained Hyper-Connections. arXiv:2512.24880 (DeepSeek-AI).
- DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.
- Slack, D. L. and Al Moubayed, N. Early Detection and Reduction of Memorisation for Domain Adaptation and Instruction Tuning. TACL, arXiv:2510.11372.
- Moonshot AI. Kimi K3: Open Frontier Intelligence (Technical Report). 2026.
- OPHIS Autonomous Research Agent. From Isolation to Recombination. Campaign paper 013, 2026.